

ProblemSet5

Shreejoyee Nath

2026-02-26

Problem Set 5: K nearest neighbours regression

1 Problem to demonstrate the utility of K nearest neighbour regression over least squares regression Consider a setting with $n = 1000$ observations. Generate

(i) x_{1i} from $N(0, 22)$ and x_{2i} from $\text{Poisson}(\lambda = 1.5)$.

(ii) ϵ_i from $N(0, 1)$.

(iii) $y_i = -2 + 1.4x_{1i} - 2.6x_{2i} + \epsilon_i$.

Split the data into train and test sets. Keep the first 800 observations as training data and the remaining as test data. Work out the following:

```
rm(list=ls())
set.seed(123)
n=1000
x1=rnorm(n,0,2)
x2=rpois(n,1.5)
e=rnorm(n)
y=-2+1.4*x1-2.6*x2+e
train.data=data.frame(y[1:800],x1[1:800],x2[1:800])
colnames(train.data)=c("y","x1","x2")
test.data=data.frame(y[801:1000],x1[801:1000],x2[801:1000])
colnames(test.data)=c("y","x1","x2")
```

1. Fit a multiple linear regression equation of y on x_1 and x_2 . Calculate test MSE.

```
fit1=lm(y~x1+x2,data=train.data)
summary(fit1)
```

```
##
## Call:
## lm(formula = y ~ x1 + x2, data = train.data)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
##	-3.0727	-0.6573	-0.0125	0.6921	3.2412

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)	
## (Intercept)	-2.07300	0.05382	-38.52	<2e-16	***
## x1	1.38207	0.01767	78.21	<2e-16	***
## x2	-2.55584	0.02768	-92.34	<2e-16	***
## ---					

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.98 on 797 degrees of freedom
## Multiple R-squared:  0.9492, Adjusted R-squared:  0.9491
## F-statistic: 7445 on 2 and 797 DF,  p-value: < 2.2e-16

y.hat.test=predict(fit1,newdata=test.data)
mse.test=mean((test.data$y-y.hat.test)^2)
mse.test

## [1] 0.998901
```

2. Fit a KNN model with $k = 1, 2, 5, 9, 15$. Calculate test MSE for each choice of k .

```
k=c(1,2,5,9,15)
library(FNN)

## Warning: package 'FNN' was built under R version 4.5.2

X.train=matrix(c(x1[1:800],x2[1:800]),byrow=F,ncol=2)
X.test=matrix(c(x1[801:1000],x2[801:1000]),byrow=F,ncol=2)
f=function(k){
  pred=knn.reg(train=X.train,test=X.test,y=train.data$y,k)
  mean((test.data$y-pred$pred)^2)
}
f(1);f(2);f(5);f(9);f(15)

## [1] 2.219793
## [1] 1.729587
## [1] 1.303978
## [1] 1.205371
## [1] 1.235776

library(caret)

## Warning: package 'caret' was built under R version 4.5.2
## Loading required package: ggplot2
## Warning: package 'ggplot2' was built under R version 4.5.2
## Loading required package: lattice

tr_kr_1=knnreg(y~x1+x2,data=train.data,k=1)
y_KNN_1=predict(tr_kr_1,test.data)
mse_knn_1=mean((test.data$y-y_KNN_1)^2)
mse_knn_1

## [1] 2.219793
```

Suppose the data in Step (iii) is generated as :

$$y_i = 1/(-2 + 1.4x_{1i} - 2.6x_{2i} + 2.9x_{1i}x_{2i}) + 3.1 \sin(x_{2i}) - 1.5x_{1i}x_{2i}^2 + \epsilon_i.$$

Work out the problems in (1) and (2). Compare and comment on the results.

```
y.new=1/(-2+1.4*x1-2.6*x2+2.9*x1*x1)+3.1*sin(x2)-1.5*x1*x2*x2+e
train.data.new=data.frame(y.new[1:800],x1[1:800],x2[1:800])
colnames(train.data.new)=c("y", "x1", "x2")
test.data.new=data.frame(y.new[801:1000],x1[801:1000],x2[801:1000])
colnames(test.data.new)=c("y", "x1", "x2")
fit3=lm(y~x1+x2,data=train.data.new)
summary(fit3)

##
## Call:
## lm(formula = y ~ x1 + x2, data = train.data.new)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -241.819   -5.150   -0.051    5.566  155.662
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -0.2369     1.0272  -0.231  0.81764
## x1           -6.3747     0.3373 -18.901 < 2e-16 ***
## x2             1.3849     0.5283   2.621  0.00892 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 18.7 on 797 degrees of freedom
## Multiple R-squared:  0.3146, Adjusted R-squared:  0.3129
## F-statistic: 182.9 on 2 and 797 DF,  p-value: < 2.2e-16

y.hat.test.new=predict(fit3,newdata=test.data.new)
mse.test.new=mean((test.data.new$y-y.hat.test.new)^2)
mse.test.new

## [1] 205.1776
```

Case 1: Linear relationship

The response variable was generated using a linear function.

This is exactly the type of relationship that multiple linear regression is designed to model.

Observations:

- The test MSE of linear regression is the lowest or among the lowest.
- KNN regression gives higher MSE compared to linear regression.
- Increasing k improves stability but still does not outperform linear regression.

Reason:

Linear regression correctly captures the true functional relationship because the true model is linear. KNN, being a non-parametric method, estimates locally and does not exploit the global linear structure efficiently. Therefore, linear regression performs better.

Conclusion for linear case:

Linear regression is superior when the true relationship is linear.

Case 2: Nonlinear relationship

The response was generated using a nonlinear function.

This function includes:

- quadratic term
- sine function
- interaction term

These cannot be captured by simple linear regression.

Observations:

- Linear regression produces very high test MSE.
- KNN regression produces significantly lower test MSE.
- Moderate values of k (such as 5, 9, or 15) typically give best results.

Reason:

Linear regression assumes a linear relationship and cannot model nonlinear patterns, resulting in poor predictions.

KNN regression is a non-parametric method and does not assume any functional form. It estimates values based on nearby observations and can adapt to nonlinear patterns.

Conclusion for nonlinear case:

KNN regression performs significantly better than linear regression when the true relationship is nonlinear.

Conclusion:

This experiment demonstrates that:

- Linear regression performs best when the true relationship is linear because it correctly models the underlying structure.
- KNN regression performs better when the true relationship is nonlinear because it does not assume linearity and adapts locally.

Therefore, KNN regression is more flexible and useful for nonlinear problems, whereas linear regression is more efficient and accurate when the true relationship is linear.